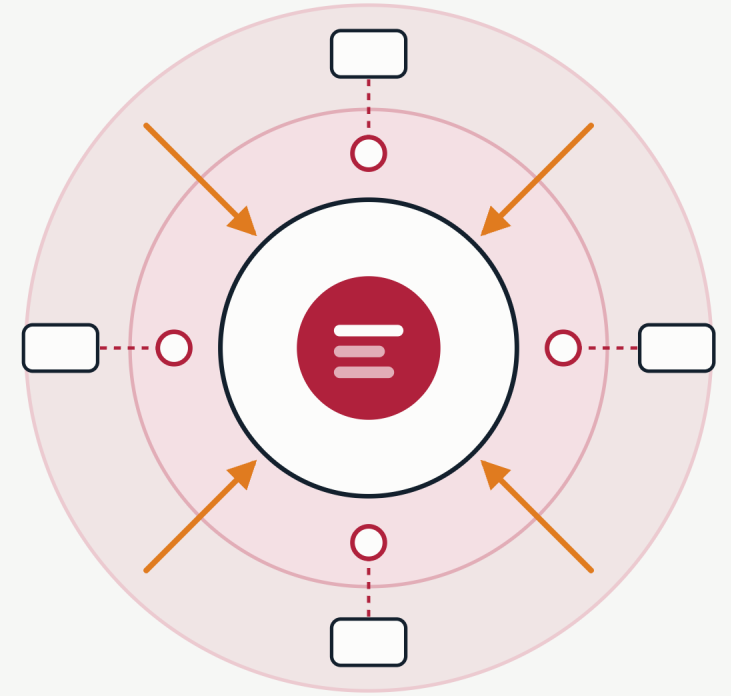


SOLID принципи и Clean Architecture

Примена SOLID принципа и Clean Architecture на границе и зависности система.



Садржај презентације

01

Основе и уговори

Порекло принципа,
предуслови и уговори
пословног језгра.

04–07

02

SRP и OCP

Један разлог за
промену и проширење
без измене
стабилног тока.

08–13

03

LSP, ISP и DIP

Заменљивост, мали
уговори и смер
зависности.

14–18

04

Clean Architecture

Смер зависности,
слојеви, ток захтева и
composition root.

19–24

05

Примена и преглед

Рефакторисање, code
review, задатак,
грешке и припрема.

25–32

Увод · улога SOLID принципа (03)

Улога SOLID принципа

Принципи локализују измене, омогућавају замену зависности и олакшавају проверу понашања.

S

Single Responsibility

један кохерентан
разлог за промену

O

Open/Closed

проширење без
измене стабилног тока

L

Liskov Substitution

заменљивост без
нарушавања
очекивања

I

Interface Segregation

мали уговори према
потреби клијента

D

Dependency Inversion

зависност од
апстракција, а не
од детаља

НАПОМЕНА

D принцип повезује SOLID са Clean Architecture, јер одређује смер зависности између слојева.

01

ПРВИ ДЕО

Основе и уговори

Одакле потичу принципи, шта је потребно за њихову примену и како уговори описују пословно језгро.

Порекло принципа

Interface

Енкапсулација

Полиморфизам

Уговори пословног језгра

Порекло принципа

Принципи су настајали током више деценија, а заједнички назив добили су знатно касније.

1987

Barbara Liskov

Предавање „Data Abstraction and Hierarchy” на OOPSLA, основа LSP-а.

1988

Bertrand Meyer

Књига „Object-Oriented Software Construction” формулише OCP.

2000

Robert C. Martin

Рад „Design Principles and Design Patterns” обједињује принципе.

≈ 2004

Michael Feathers

Уводи скраћеницу SOLID за пет Martin-ових принципа.

2012

Robert C. Martin

Текст „The Clean Architecture”, а 2017. и истоимена књига.

Сродни приступи су Hexagonal Architecture (Alistair Cockburn, 2005) и Onion Architecture (Jeffrey Palermo, 2008). Сви деле исто правило: зависности су усмерене ка пословном језгру.

Предуслови за примену SOLID принципа

Уговори, енкапсулација, полиморфизам и одговорност чине основу примене принципа.

Појмови

Interface дефинише очекивано понашање.

Енкапсулација чува инваријанте.

Полиморфизам омогућава заменљиве имплементације.

Апстрактна класа има смисла када постоји стварна заједничка основа.

Практичан ефекат

код описује намеру

одговорност је видљива

промене су локализоване

тестови лакше прате понашање

Уговори пословног језгра у примеру Logger–Blogger

Доменски слој описује способности система без зависности од складишта и корисничког интерфејса.

C# • Domain

```
1  public interface IZapisiServis
2  {
3      Result<ZapisNaSajtu> Objavi(ZapisNaSajtu zapis);
4      Result<IReadOnlyList<ZapisNaSajtu>> PregledZapisa();
5  }
6
7  public interface IBazaPodataka
8  {
9      TabeleBazaPodataka Tabele { get; }
10     bool SacuvajPromene();
11 }
```

НАПОМЕНЕ

Уговор описује стабилну потребу пословног језгра.

Domain не зависи од Database или Presentation пројекта.

Заменљивост долази из јасних уговора, а не из назива слојева.

02

ДРУГИ ДЕО

SRP и OCP

Како одговорност класе и место варијације одређују где ће се промена десити.

Single Responsibility

ZapisiServis

Open/Closed

switch наспрам уговора

Заменљиво складиште

Single Responsibility Principle

Класа може имати више метода када све припадају истој одговорности.

Проблем није број линија кода, већ чињеница да се иста класа мења из више међусобно независних разлога.

ФОРМУЛАЦИЈА • MARTIN, 2017

Модул треба да одговара једном и само једном актеру, односно једној групи корисника која тражи промене.

НАПОМЕНА

SRP подстиче кохерентне компоненте са јасном одговорношћу.

SRP у сервису Logger–Blogger

Сервис оркестрира објаву преко уговора, без одговорности интерфејса, базе и логовања.

C# • ZapisiServis.cs

```
1  public sealed class ZapisiServis : IZapisiServis
2  {
3      private readonly IBazaPodataka _baza;
4      private readonly ILogger _logger;
5      // конструктор прима оба уговора
6      public Result<ZapisNaSajtu> Objavi(ZapisNaSajtu zapis)
7      {
8          _baza.Tabele.Zapisi.Add(zapis);
9          _baza.SacuvajPromene();
10         _logger.Info("Objavljen zapis");
11         return Result<ZapisNaSajtu>.Ok(zapis);
12     }
13 }
```

НАПОМЕНЕ

Сервис оркестрира сараднике кроз уговоре.

Начин чувања и логовања остаје спољни детаљ.

Ако се промени logger, пословни ток не мора да се мења.

Open/Closed Principle

Апстракције се уводе за познате и оправдане варијације понашања.

Ако стабилан део система стално добија нови switch по типу понашања, променљиво понашање треба сместити иза уговора.

ФОРМУЛАЦИЈА • MEYER, 1988

Софтверски ентитети треба да буду отворени за проширење, а затворени за измену.

НАПОМЕНА

OCP омогућава проширење понашања без измене стабилног тока.

Од switch-а до уговора

Илустрација: извоз записа у више формата.

ПРЕ • SWITCH ПО ФОРМАТУ

```
string Izvezi(ZapisNaSajtu z, string f) =>
    f switch
    {
        "json" => UJson(z),
        "md" => UMarkdown(z),
        _ => throw new NotSupportedException()
    };

```



ПОСЛЕ • УГОВОР

```
public interface IIzvozZapisa
{
    string Format { get; }
    string Izvezi(ZapisNaSajtu zapis);
}
// нови формат је нова класа
sealed class MarkdownIzvoz : IIzvozZapisa

```

⚠ Сваки нови формат мења исту стабилну методу.

✓ Нови формат се додаје, а постојећи код се не мења.

OCP и заменљиво складиште

Нова имплементација складишта додаје се без измене пословног кода.

C# • Domain и Infrastructure

```
1  public interface IBazaPodataka
2  {
3      TabeleBazaPodataka Tabele { get; }
4      bool SacuvajPromene();
5  }
6
7  public sealed class InMemoryBazaPodataka : IBazaPodataka
8  {
9      public TabeleBazaPodataka Tabele { get; } = new();
10     public bool SacuvajPromene() => true;
11 }
```

НАПОМЕНЕ

Стабилан сервис користи IBazaPodataka.

JSON, in-memory или други provider су технички детаљи.

Проширење се дешава додавањем имплементације.

03

ТРЕЋИ ДЕО

LSP, ISP и DIP

Принципи који откривају неодговарајуће уговоре и погрешан смер зависности.

Дефинисање граница

Заменљивост подтипа

Подела улога

Инверзија зависности

LSP, ISP и DIP у дефинисању граница

Принципи откривају неодговарајуће уговоре и погрешан смер зависности.

Принцип	Проблем	Смер корекције
LSP	подтип мења очекивање	редизајнирати уговор
ISP	клијент зависи од вишка операција	поделити улоге
DIP	use-case креира SQL или HTTP детаљ	зависити од уговора

НАПОМЕНА

Наредна три слајда приказују сваки принцип на примеру из Logger–Blogger домена.

Liskov Substitution Principle

Подтип сме да проширује понашање, али не сме да наруши оно што уговор обећава.

LISKOV И
WING, 1994

$\varphi(x)$ за свако $x : T \Rightarrow \varphi(y)$ за свако $y : S$, где је S подтип од T

C# • пример нарушавања LSP-a

```
1 public sealed class ArhivaBazaPodataka : IBazaPodataka
2 {
3     public TabeleBazaPodataka Tabele { get; } = new();
4     public bool SacuvajPromene() =>
5         throw new NotSupportedException();
6 }
```

НАПОМЕНЕ

- Уговор обећава резултат true или false.
- Подтип баца изузетак и руши ZapisiServis.
- Корекција: посебан уговор само за читање.

Interface Segregation Principle

Клијент зависи само од операција које стварно користи.

ПРЕ • ШИРОК УГОВОР

IUpravljanjeZapisima

Objavi(zapis)

PregledZapisa()

IzveziUJson(putanja)

ObrisiSve()

Приказ листе записа зависи и од брисања и од извоза.

ПОСЛЕ • УГОВОРИ ПО УЛОГАМА

IObjavaZapisa

Objavi • користи форма за објаву

IPregledZapisa

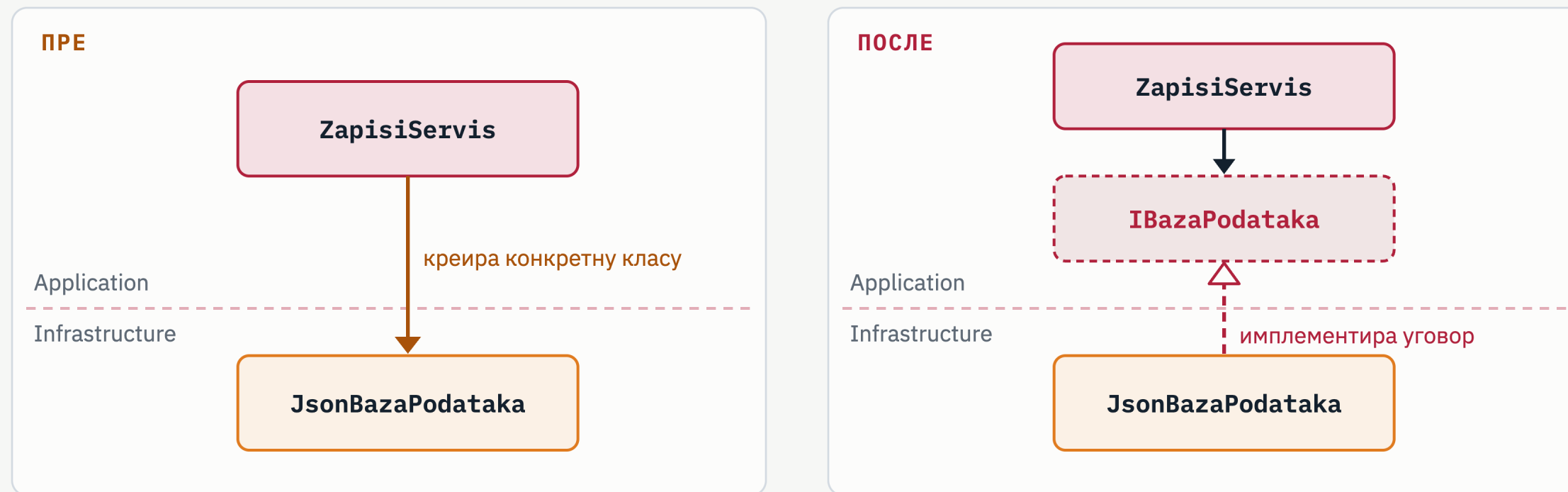
PregledZapisa • користи приказ листе

IAdministracijaZapisa

IzveziUJson, ObrisiSve • користи администрација

Dependency Inversion Principle

Политика зависи од апстракције, а технички детаљ имплементира ту апстракцију.



04

ЧЕТВРТИ ДЕО

Clean Architecture

Слојеви система и правило по коме зависности увек показују ка пословном језгру.

Смер зависности

Одговорности слојева

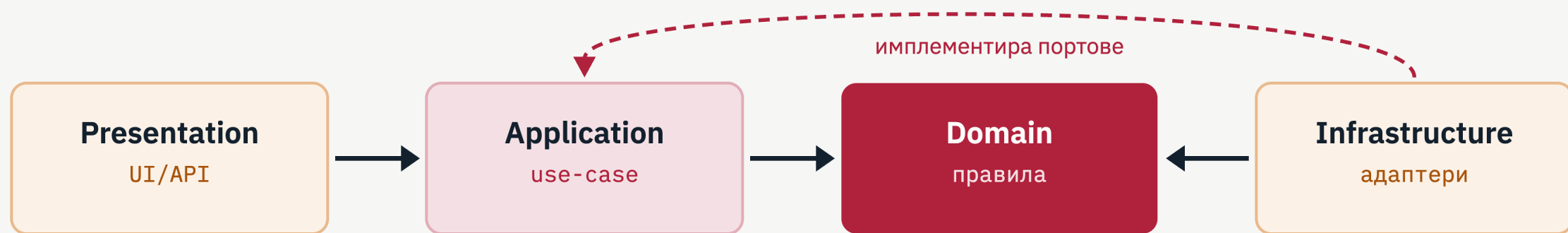
Ток захтева

Composition root

Студија случаја

Смер зависности у Clean Architecture

Називи пројеката могу да се разликују. Смер зависности остаје јасан и проверљив.



Tests **провера** • зависе од слоја који проверавају и нису део ланца

НАПОМЕНА

Пословно језгро остаје независно од радног оквира, базе података, HTTP-а и корисничког интерфејса.

Одговорности архитектонских слојева

Квалитет архитектуре процењује се по јасноћи одговорности и смеру зависности.

01 Domain

ентитети, инваријанте и стабилна правила



02 Application

случајеви употребе и портови



03 Infrastructure

база, датотеке и спољни сервиси



04 Presentation

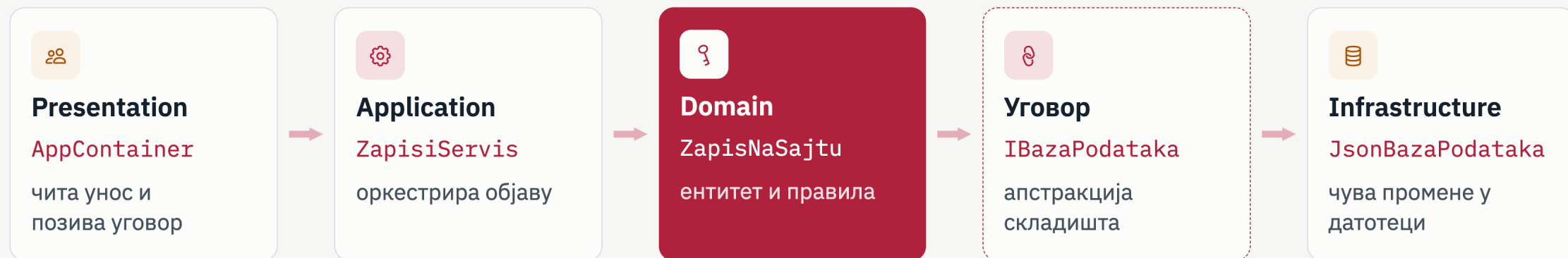
превод захтева у позив use-case-a



Плаве ознаке означавају пословно језгро, а наранџасте спољне детаље који се могу заменити.

Пут једне објаве кроз слојеве

Позив иде од корисничког интерфејса ка језгру, а складишту стиже само преко уговора.



НАПОМЕНА

Ток позива не мора да прати смер зависности. JsonBazaPodataka се позива, али зависи од уговора, а не обрнуто.

Composition root у примеру Logger–Blogger

Једно место бира конкретне имплементације, а пословни код зависи од уговора.

```
C# • Presentation • улазна тачка апликације
1 IBazaPodataka baza = new JsonBazaPodataka("data.json");
2 ILogger logger = new FileLogger("app.log");
3
4 IZapisiServis zapisi = new ZapisiServis(baza, logger);
5 IConsoleIO io = new ConsoleIO();
6
7 var app = new AppContainer(zapisi, io);
8 app.Run();
```

НАПОМЕНЕ

Presentation саставља граф зависности.

Domain и сервиси не креирају JSON базу или FileLogger.

DIP омогућава замену детаља без промене језгра.

Logger–Blogger као студија случаја

Пример служи за анализу одговорности, уговора и зависности.

01

Пословно језгро

правила без инфраструктурних
детаља

ZapisNaSajtu

IZapisiServis

IBazaPodataka

ZapisiServis

02

Adapter-и

конкретна база, датотеке, логовање

JsonBazaPodataka

InMemoryBazaPodataka

FileLogger

ConsoleIO

03

Composition root

место где се саставља
граф зависности

Presentation

AppContainer

05

ПЕТИ ДЕО

Примена и преглед

Како препознати потребу за рефакторисањем, проверити дизајн кроз code review и избећи механичку примену принципа.

Рефакторисање

Питања за code review

Задатак на вежби

Критеријум примене

Типичне грешке

Припрема

Разлози за рефакторисање

Декомпозиција се спроводи када смањује спрегу и локализује измене.



Лош сигнал

класа мења податке, шаље email и логује

метода има више неповезаних праваца

тест мора да зна приватне детаље

ново понашање тражи измену стабилног центра



Добар сигнал

једна реченица описује одговорност

сарадници су уговори

промена погађа очекивано место

тестови проверавају понашање

Питања за преглед кода

Питања усмеравају анализу последица дизајнерских одлука.

Питање	Шта открива
Да ли модул има један разлог за промену?	SRP
Да ли нова варијанта мења стабилан ток?	OCP
Да ли имплементација поштује уговор?	LSP
Да ли је интерфејс шири него што клијенту треба?	ISP
Да ли политика зависи од техничког детаља?	DIP

Задатак на вежби

SOLID принципи примењују се на конкретан код и образлажу кроз последице измене.

- | | | |
|----|---|-------------|
| 01 | Пронаћи класу са најмање две неповезане одговорности. | SRP |
| 02 | Описати њену тренутну улогу једном реченицом. | опис |
| 03 | Предложити раздвајање и нове границе. | дизајн |
| 04 | Образложити које будуће промене постају локализоване. | образложење |
| 05 | Проверити да ли је смер зависности остао исправан. | DIP |

Критеријум примене

Последица промене треба да остане локализована и предвидљива.

1

ЈЕДНА ЈАСНА ПРОМЕНА

Када се промени једно пословно правило или технички детаљ, треба да се мења мали и очекиван део система.



Локализована промена: једна промена, једно очекивано место.



Расута промена: једна промена захтева измене у много класа.

Типичне грешке

Механичка примена принципа ствара непотребне апстракције и сложеност.

01 Превише интерфејса
апстракције без реалне варијабилности

02 Погрешно наслеђивање
подтип не поштује уговор

03 Anemic domain
сва правила су у сервисима

04 Framework у језгру
Domain зависи од инфраструктуре

НАПОМЕНА

Принцип се примењује када решава постојећи проблем, а не зато што је познат.

SOLID и Clean Architecture штите разумљивост, заменљивост и проверљивост система.

Добар дизајн није онај који има највише слојева, већ онај у коме се одговорност, смер зависности и последица промене могу јасно објаснити.



Наредна вежба разматра положај пословне логике у доменском и апликационом слоју.

Припрема за наредну вежбу

Наредна вежба разматра положај пословне логике и случајева употребе.

01

Изабрати један use-case у свом домену.

02

Означити Domain, Application и Infrastructure одговорности.

03

Пронаћи једну класу са нејасном одговорношћу.

04

Предложити refactoring са образложењем.

05

Припремити пример за code review.

🔍 Питања за преглед кода са слајда 27 користити као контролну листу.